
LOCAL SEARCH & OPTIMIZATION

INSTRUCTOR: MOAYAD ALNAMMI

malnammi.github.io

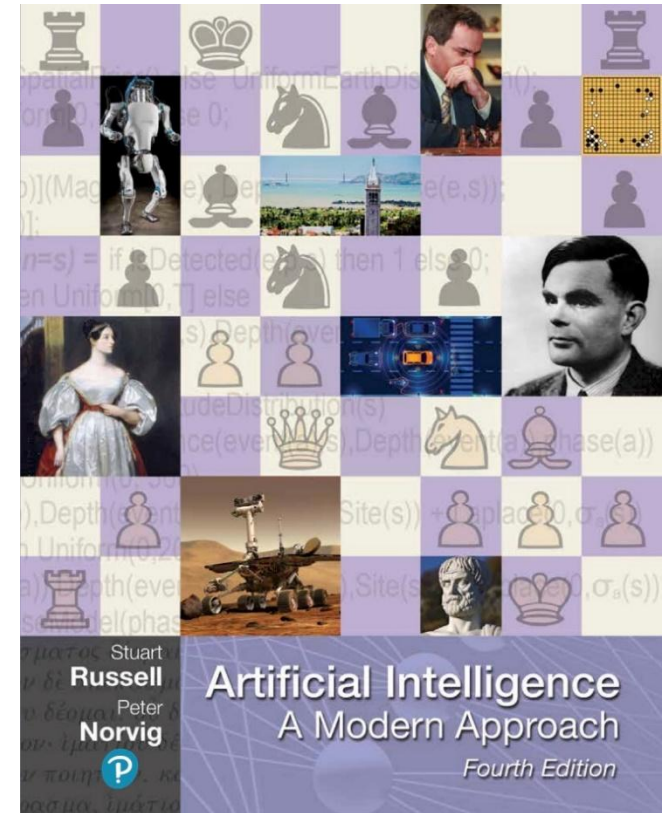
Notice

- Slides prepared by Moayad Alnammi – malnammi.github.io
- Some figures adapted from CS188: Introduction to Artificial Intelligence, UC Berkeley (Dan Klein, Pieter Abbeel) — ai.berkeley.edu
- Some figures are obtained from the [AIMA Book](#)
- Some content adapted from Dr. ElSayed El-Alfy

This deck is published at malnammi.github.io under CC BY-NC-SA 4.0. Reuse and adaptation require attribution to the author and the sources above; commercial use is not permitted; adaptations must carry the same license.

Resources & Reading

- Textbook website: <http://aima.eecs.berkeley.edu/>
- Read Chapter 4.1, skim 4.2 in AIMI 4/E



Outline

- Introduction to Local Search Methods
- Hill-climbing search
- Simulated annealing
- Evolutionary algorithms
- Issues with local search
- Summary

Local Search

Local Search Intro [1]

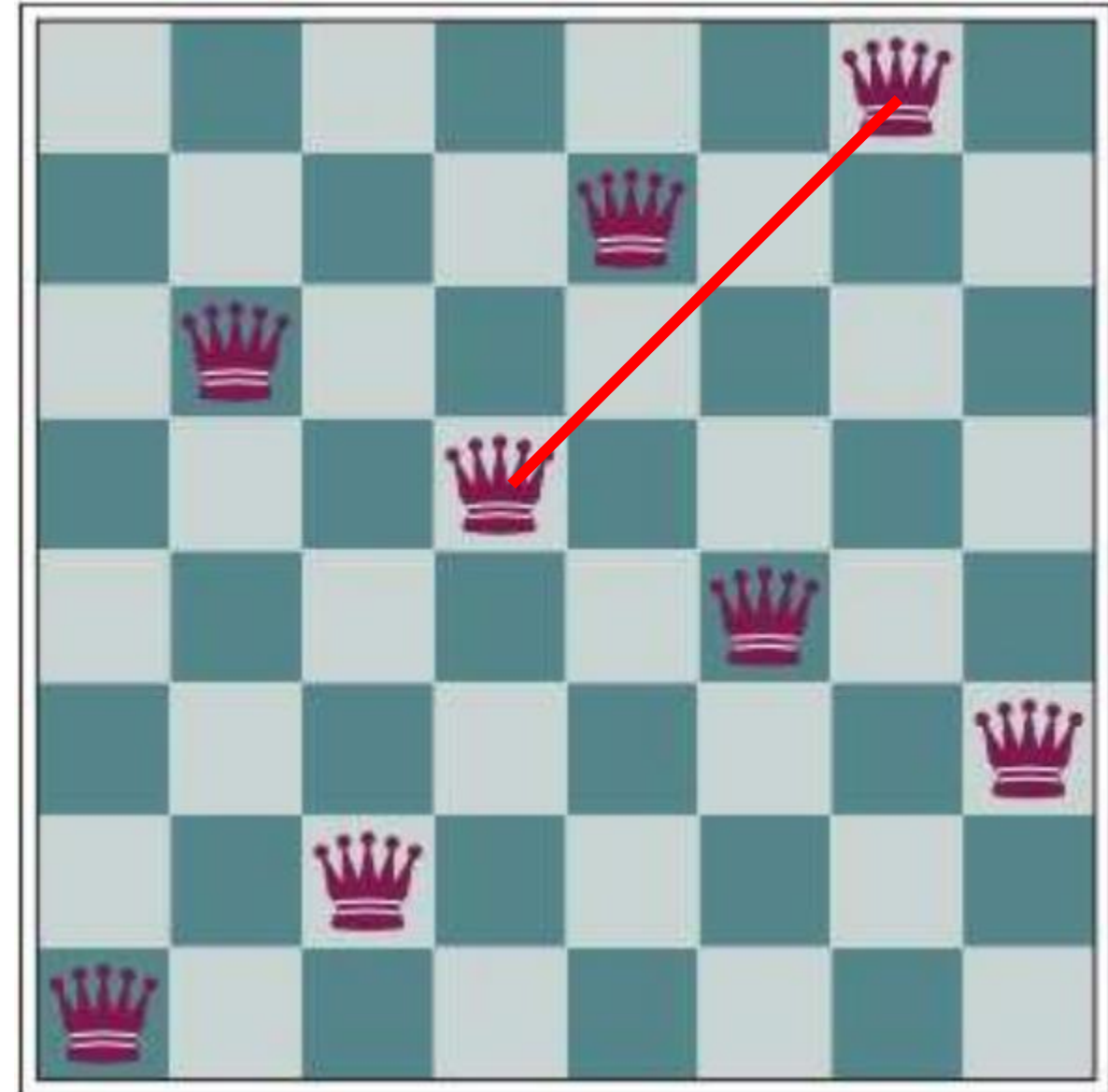
- In previous topics, we discussed problems where the goal was to find a solution path; i.e. a sequence of actions that lead from the initial state to the goal.
- For some problems, the sequence of actions (solution path) is not important, but rather, finding the goal state is important. That is, we want to find a state with a specific configuration.
- Thus, the the problem is to find a state with configuration satisfying some constraints.

Find state s^* such that $conditions(s^*) = true$. Note there may be more than one such s^* .

- Examples where goal state is important, but path to reach is not:
 - circuit design/floor layout: position components on grid s.t. constraints satisfied.
 - Job/airline/course scheduling: schedule jobs/flights/course in timetable s.t. constraints satisfied.
 - Packing problems: position objects into an area/volume s.t. constraints satisfied.
 - 8-queens problem (see next slide)
 - Many problems from previous topic (uninformed and informed searching)

Example problem – 8-queens

- 8-queens problem: position 8 queens on the board such that no two queens can attack each other. (A queen attacks any piece in the same row, column, or diagonal)
- Notice the goal here is to find the satisfying configuration: positioning of the 8-queens with no attacks.
- Notice in the picture, queens in column 4 and column 7 are attacking each other, so this configuration is not satisfying.



Local Search Intro [3]

- Local search algorithms operate by maintaining a current state (or set of current states) in memory.
- The “local” searching is done by **selecting from among the neighbor-states the most promising**. Keep doing this, moving from current-state to neighboring states until a desired goal state is reached.
- Needs a way to select from among neighboring states: use **objective/fitness function $f(s)$** that assigns a value to a state s . Select the neighbor with the best $f(s)$.
- We can then say that the goal state is the state with optimum $f(s)$ value.
- Optimality of local search algorithms:
 - Depends on the algorithm, but most local search algorithms are not guaranteed to find *optimal* state; i.e. state with optimum objective value.
- Advantages:
 - Just maintain the current state in memory; no need to keep track of path. (low memory)
 - If algorithm is not guaranteed to find optimal solution, can still find reasonably *good* solutions; i.e. states that have *good* objective value but may not be the optimal objective value.
- Suitable when state space is massive or infinite.

Local Search Intro [4]

- Local search algorithms use an objective/fitness function to assign a value to states. The goal state is the one with the optimum $f(s)$.
- Thus, local search algorithms can be used to find the state with max (or min) objective value.

$$s^* = \arg \max_{s \in S} f(s)$$

$$s^* = \arg \min_{s \in S} f(s)$$

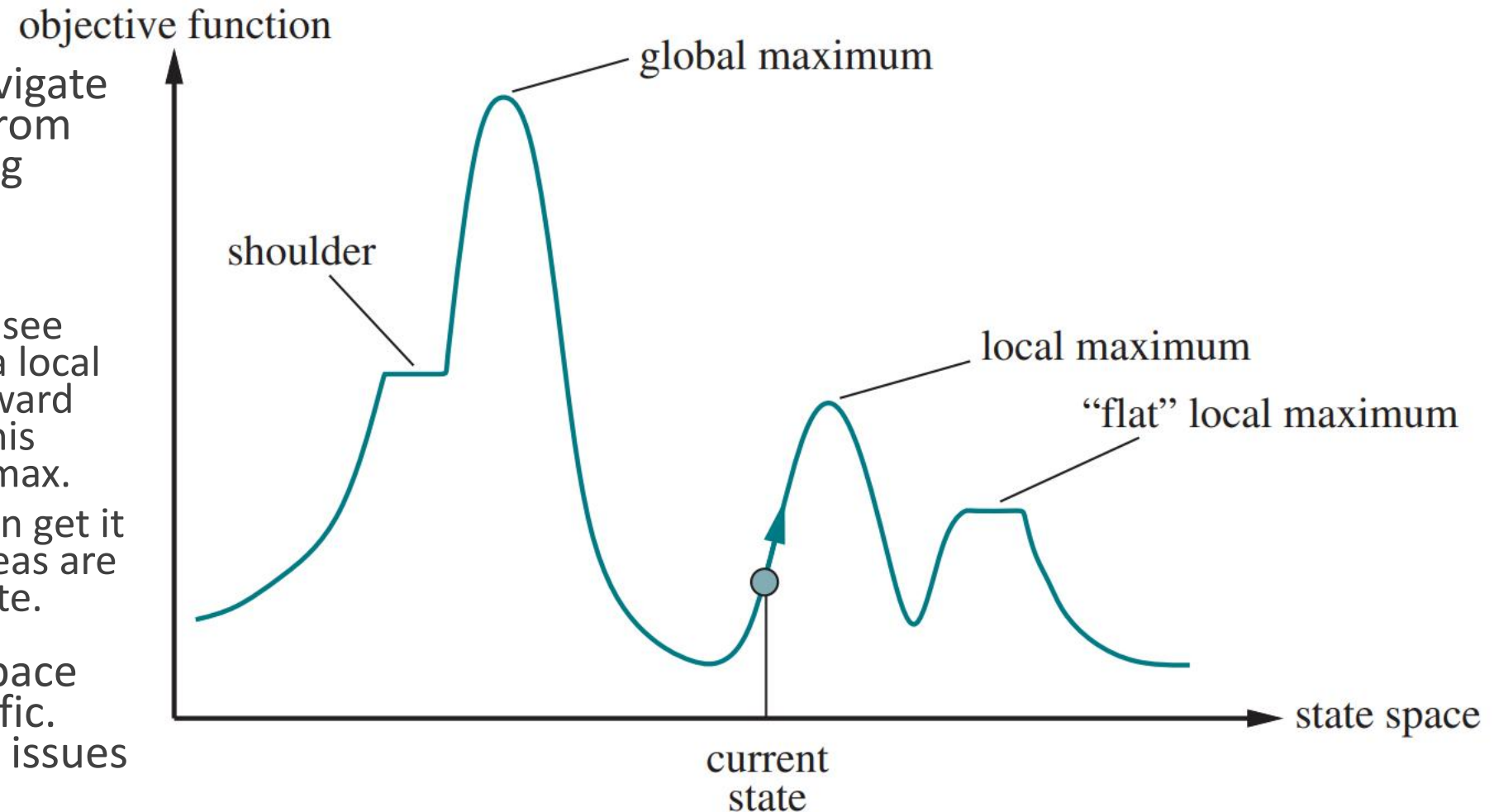
- If the goal is to find **global maximum** state (argmax), then this is called hill-climbing.
- If the goal is to find **global minimum** state (argmin), then this is called hill-descent (also known as gradient descent).
- Note that negative the objective function in argmax is equivalent to argmin $f(s)$, and vice-versa.

$$\arg \max_{s \in S} -f(s) = \arg \min_{s \in S} f(s)$$

Local Search Intro [5] – state-space landscape

- Suppose we can project the states on the x-axis, and their objective function value on the y-axis. This gives rise to the state-space landscape. Assume the goal is to find state with maximum value.

- Local search algorithms navigate this landscape by moving from current state to neighboring state.
- Some issues:
 - Local view limits ability to see global max. In the figure, a local algorithm would move toward the *better* neighbor, but this would lead it to the local max.
 - Flat and shoulder areas can get it stuck. Neighbors in flat areas are no better than current state.
- Note that resulting state-space landscape is problem specific. Some problems have more issues than others.



Local Search Intro [6] – state-space landscape

- If the goal is to find the minimum, landscape can have regions of local minima, flat areas, shoulders, and global minima.

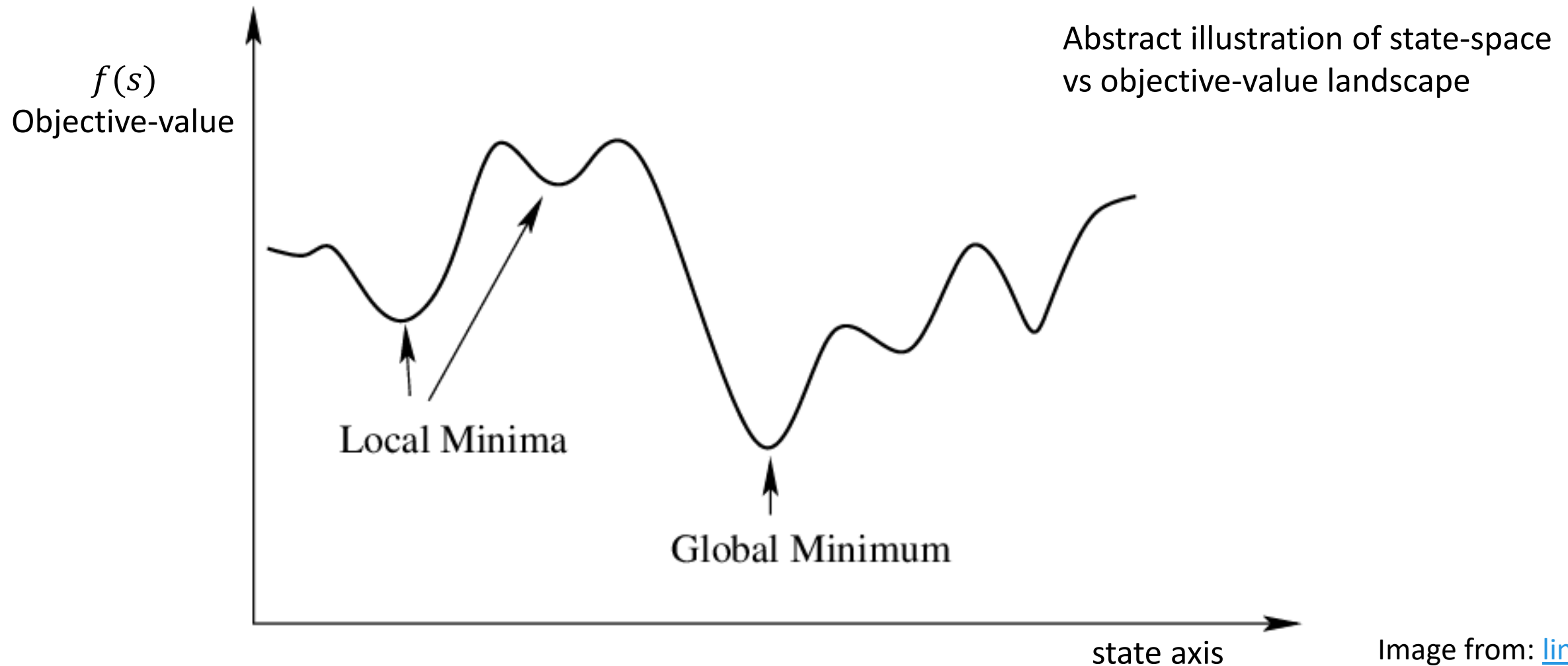


Image from: [link](#)

Local Search Intro [7] – Algorithms

- Simple local search algorithm: Random-walk
 - From the current state, randomly (uniform-random) select from among the neighbor-states.
 - Optimality? Yes, it is optimal with probability 1. Eventually, if run for long enough it will find the goal state (optimum value).
 - Very inefficient, especially for large state-spaces.
- This class we will discuss the following local search algorithms:
 - Hill-climbing search (greedy local search)
 - Simulated annealing
 - Local Beam search and stochastic beam search
 - Evolutionary algorithms (genetic algorithm)

Side-topic: Sampling Basics – How to sample from a discrete distribution

- To sample a discrete random variable C :
 - Step 1: Take the outcomes of C and their probabilities, and associate each outcome with an interval in the range $[0, 1)$. See how this is done for the example given below.
 - Step 2: Sample u from a uniform distribution over $[0, 1)$. You can use `np.random.uniform()`.
 - Step 3: See where the value of u falls into the intervals defined in step 1. Grab the outcome that u falls into

Example:

C	$P(C)$
red	0.6
green	0.1
blue	0.3

$0.0 \leq u < 0.6, \rightarrow C = \text{red}$

$0.6 \leq u < 0.7, \rightarrow C = \text{green}$

$0.7 \leq u < 1.0, \rightarrow C = \text{blue}$



Examples of sampling outcomes for C :

- If `np.random.uniform()` returns $u = 0.83$, then what is sample for C ? $C = \text{blue}$
- If `np.random.uniform()` returns $u = 0.65$, then what is sample for C ? $C = \text{green}$
- If `np.random.uniform()` returns $u = 0.1$, then what is sample for C ? $C = \text{red}$

Hill-climbing Search

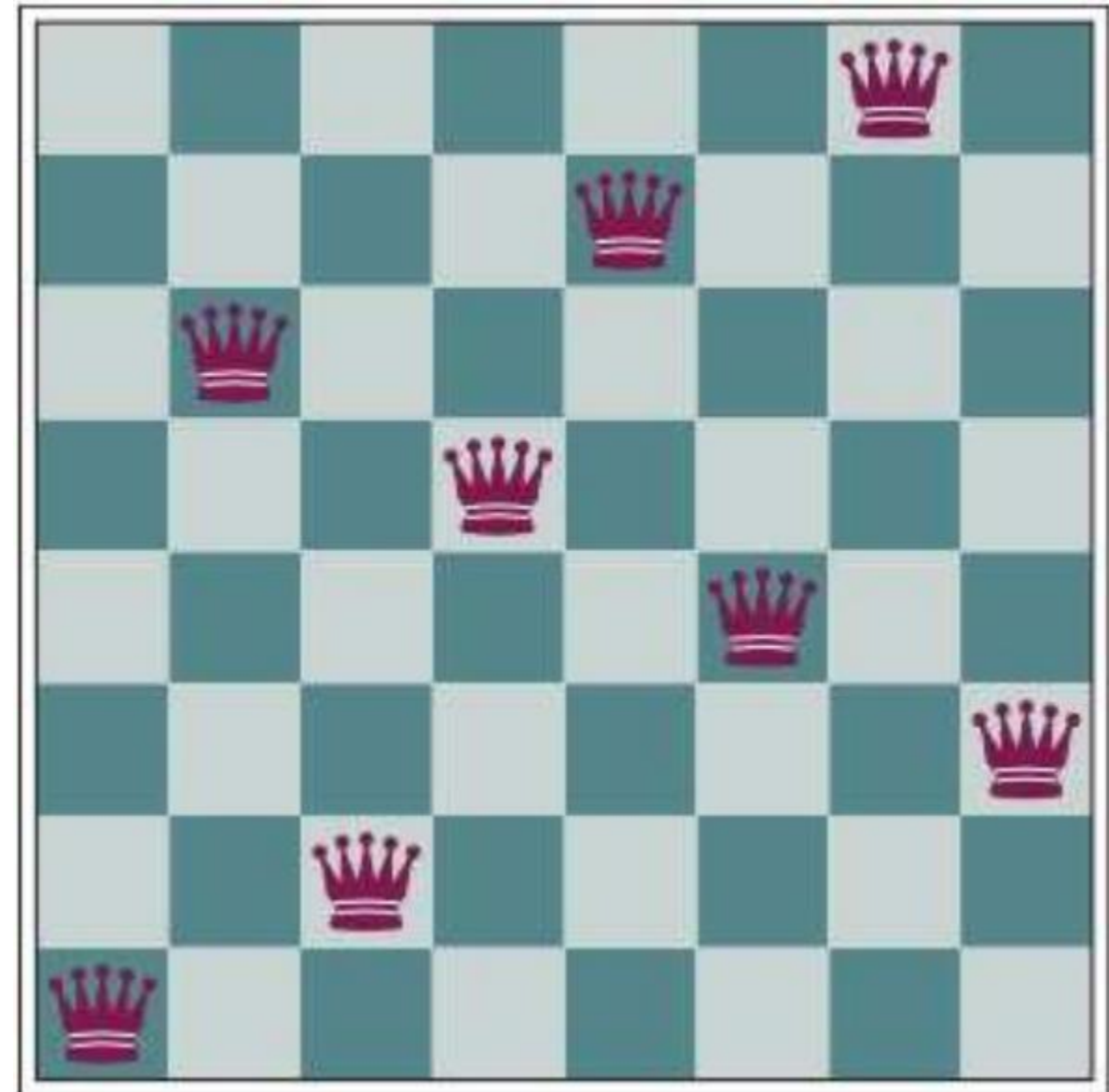
Hill-climbing search [1]

- Also known as greedy local search.
- Keeps track of a single state in memory. Randomly initialize the start state. From the current state, move to the best neighbor based on an objective value.
- Stop when no neighbor is better than current state.
- Optimal? No, not guaranteed to find global optimum.

function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum
current $\leftarrow$ *problem*.INITIAL ← Initial state can be randomly initialized
while *true* **do**
 neighbor $\leftarrow$ a highest-valued successor state of *current*
 if VALUE(*neighbor*) $\leq$ VALUE(*current*) **then return** *current*
 current $\leftarrow$ *neighbor*

Hill-climbing search [2] – 8-queens problem

- 8-queens problem: position 8 queens on the board such that no two queens can attack each other. (A queen attacks any piece in the same row, column, or diagonal.)
- # states = # of ways to position 8 queens on 64-chess board = $64 \cdot 63 \cdot 62 \cdot 61 \cdot 60 \cdot 59 \cdot 58 \cdot 57 = \sim 1.78 \cdot 10^{14}$
- A better formulation is to consider only states with the 8-queens already positioned along columns.
- 8 possible columns with 8-queens, then states differ by how they position each queen on a column. # states = $8^8 = 16,777,216$
- How to generate neighbors from current state?
This is a design choice.
 - One option: a neighbor/successor state is reached by re-positioning a single queen along a single column. # successors = $8 \cdot 7 = 56$



State = (1 6 2 5 7 4 8 3)

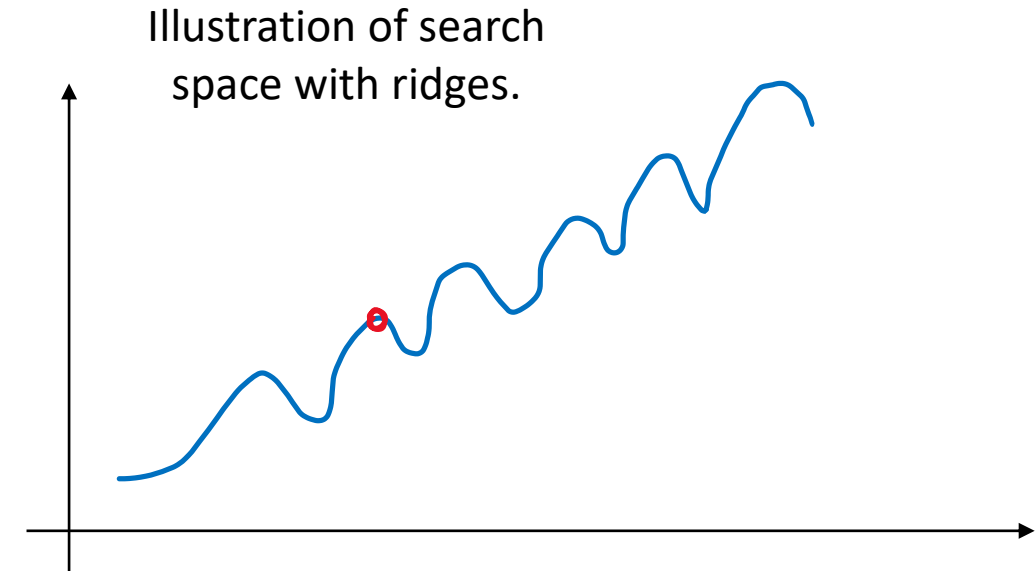
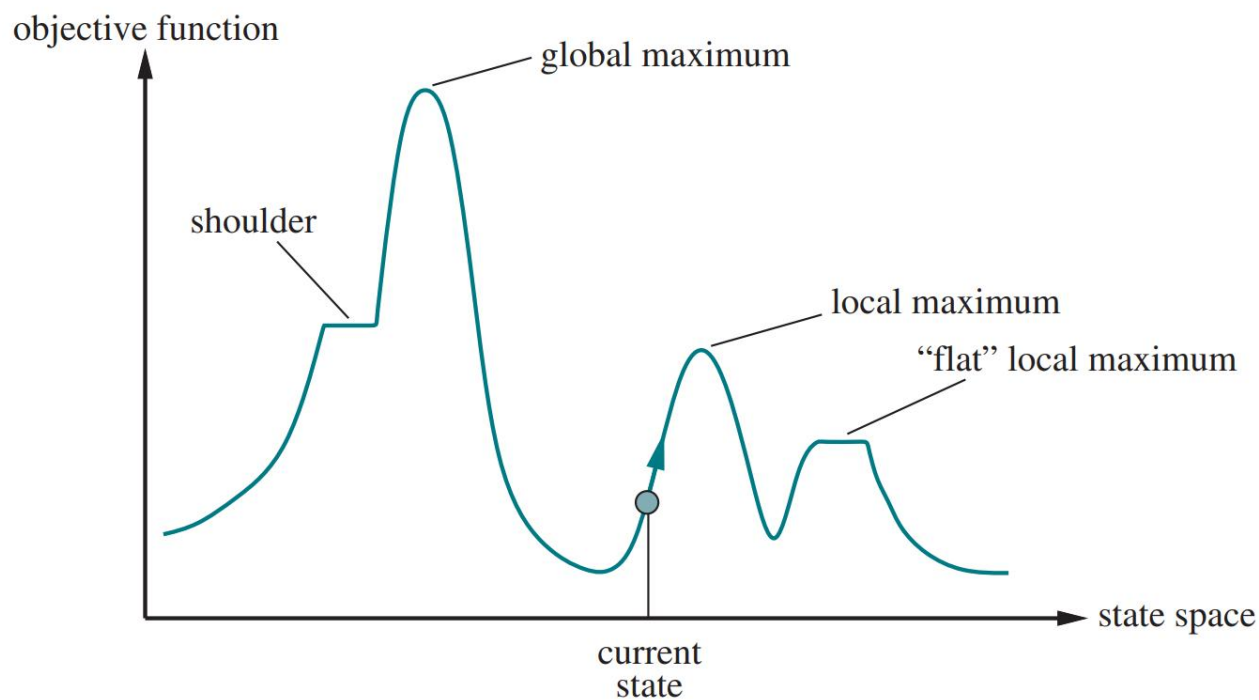
18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♙	13	16	13	16
♙	14	17	15	♙	14	16	16
17	♙	16	18	15	♙	15	♙
18	14	♙	15	15	14	♙	16
14	14	13	17	12	14	12	18

Numbers reflect the number of attacking pairs for *neighbors* of current state.

- What is a good objective function for this state formulation? We can use a cost/heuristic function.
- A reasonable cost function is the number of attacking pairs. Note a minimization problem.
- For the goal state, this should be 0 (no attacking pairs).
- For given state in figure, it shows the cost for the $8*7 = 56$ successors.

Issues with hill-climbing [1]

- We discussed this earlier:
 - Local optima: depending on start state, can lead to nearest local optima which is not global optima.
 - Plateaus: shoulders and flat areas where no neighbors are better.
 - Ridges: sequence of local optimas that are not directly connected, and actions from those local optimal lead to worse states.



Issues with hill-climbing [2]

- How to improve chances of success?
 - Allow limited **sideways** moves: helps to escape shoulders. The limit is to stop from wandering forever in true flat areas.
 - **Random-restarts**: randomly initialize the start state, run hill-climbing. If not successful, randomly initialize start state again, and run hill-climbing. Keep trying until optimal solution is found. Optimal with probability 1.
- For 8-queens problem, hill-climbing with no sideways moves (stats from book):
 - Succeeds w/ prob. 0.14 (how to get this number?)
 - Average number of steps per trial: 4 when succeeding, 3 when getting stuck
 - Expected number of restarts required until we see success: $1/0.14 = \sim 7$ restarts (note this is from [geometric distribution](#) expectation formula)
 - Expected total number of steps needed with random restarts: $3*6$ (fails) + $4*1$ (success) = ~ 22 moves
- Allowing 100 sideways moves:
 - Succeeds w/ prob. 0.94
 - Average number of steps per trial: 21 when succeeding, 65 when getting stuck
 - Expected number of restarts required until we see success: $1/0.94 = \sim 1.06$ restarts
 - Expected total number of steps needed: $65*0.06$ (fails) + $21*1$ (success) = ~ 25 moves

Simulated Annealing

Simulated annealing [1]

- Random walk algorithm will find global optimum eventually, but can run for very long time.
- Hill-climbing is not guaranteed to find global optimum, but is computationally fast.
- Combine both to hopefully achieve speed and completeness. **Simulated annealing** does this.
- Annealing is a metallurgy process to harden metals by heating them then cooling them down slowly.
- Basic idea:
 - Use a *temperature* value to control decision-making.
 - Allow *bad* moves occasionally, depending on the *temperature* and how bad the move is.
 - High temperature → more bad moves allowed, shake the system out of its local optimum.
 - Gradually reduce temperature according to some schedule.

Simulated annealing [2]

- Note this pseudocode assumes finding the global minima, so VALUE is a cost function. Notice deltaE reflects this. If trying to find global maxima, need to flip deltaE's terms.

function SIMULATED-ANNEALING(*problem, schedule*) **returns** a solution state

current $\leftarrow$ *problem*.INITIAL

for $t = 1$ **to** ∞ **do**

$T \leftarrow \text{schedule}(t)$

Scheduler for temperature T. Starts with high temperature, then reduces over iterations.

if $T = 0$ **then return** *current*

next $\leftarrow$ a randomly selected successor of *current*

Uniform-random select neighbor

$\Delta E \leftarrow \text{VALUE}(\text{current}) - \text{VALUE}(\text{next})$

Compute how "bad" this neighbor is compared to current

if $\Delta E > 0$ **then** *current* $\leftarrow$ *next*

If there is improvement, then always accept this move.

else *current* $\leftarrow$ *next* only with probability $e^{\Delta E/T}$



If no improvement, then accept with prob. that decreases as deltaE worsens (very bad moves have lowered prob.).

Also, as T decreases over iterations, the prob. of acceptance goes down. Bad moves are more likely to be allowed at start of algorithm.

Simulated annealing [3]

$$e^{\Delta E/T}$$

If we fix deltaE for a very bad move, we see that as T decreases, prob of accepting bad move decreases.

T	VALUE(current)	VALUE(next)	deltaE	prob
100	10	100	-90	4.07E-01
50	10	100	-90	1.65E-01
10	10	100	-90	1.23E-04
5	10	100	-90	1.52E-08
1	10	100	-90	8.19E-40
0.8	10	100	-90	1.39E-49
0.4	10	100	-90	1.92E-98
0.01	10	100	-90	0.00E+00

If we fix T, we see that as we vary deltaE (very bad to not-so bad move), prob of accepting very bad move is less than not-so bad move.

T	VALUE(current)	VALUE(next)	deltaE	prob
5	10	100	-90	1.52E-08
5	10	80	-70	8.32E-07
5	10	60	-50	4.54E-05
5	10	40	-30	2.48E-03
5	10	20	-10	1.35E-01

Simulated annealing [3] – properties

- Guaranteed to find optimal?
- If scheduler decreases T to 0 slowly enough, then will find optimum with probability 1.
- This is a nice theoretical guarantee, but how slow?
 - Depends on the problem. Might need to be *exponentially* slow.
- Recall the random-restart hill-climbing is also guaranteed to find local optimum and can be fast.
- Simulated annealing used in VLSI (very large-scale integrated-circuit) design and factory scheduling.

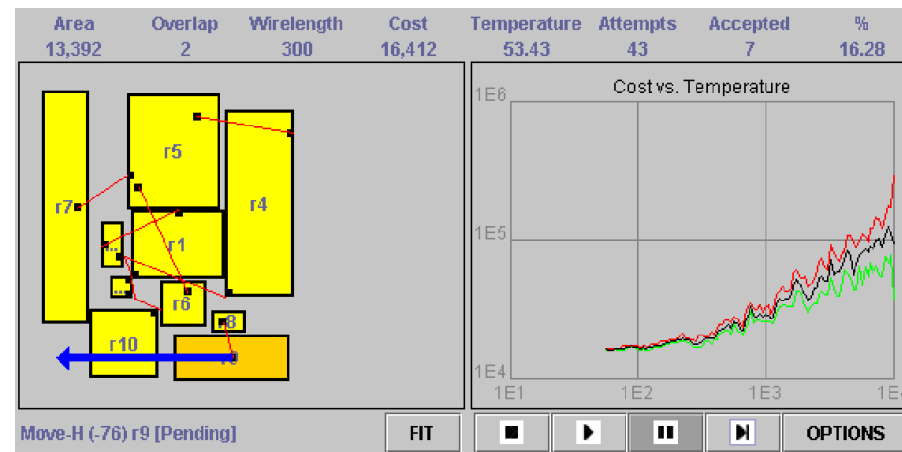
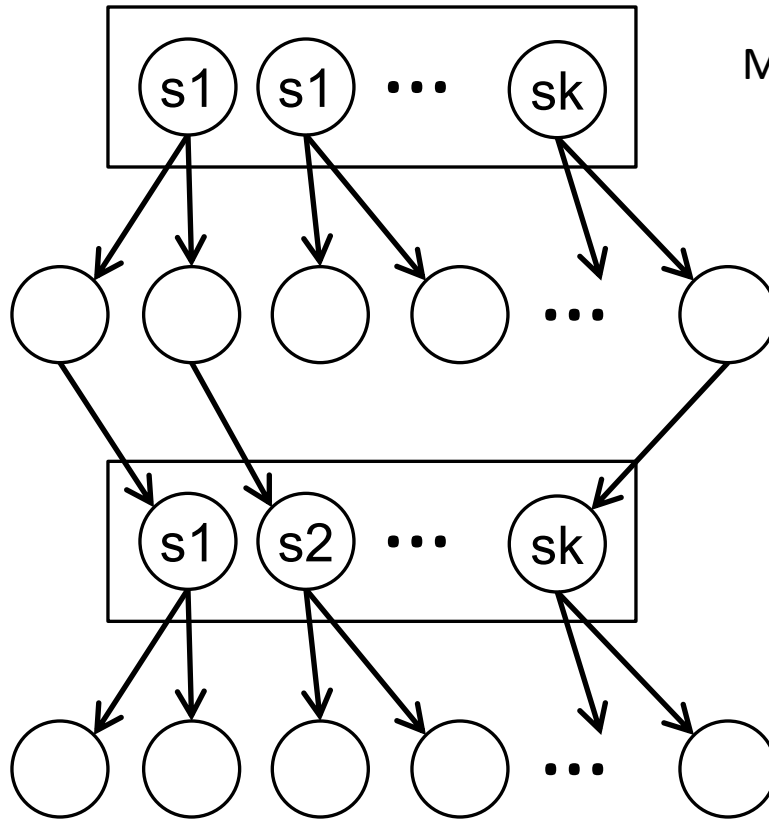


Figure 5 – Simulated Annealing Placement Applet

Circuit design. Image from [link](#)

Local Beam Search

Local Beam Search [1]



Maintain k random initial states

Generate their children

Select the k best children (in stochastic beam search select with prob. Proportional to value).

Repeat...

Local Beam Search [2]

- Basic idea:
 - Maintain k states in memory initialized randomly.
 - For each iteration
 - Generate ALL successors of the k current states: $S_1, S_2, \dots, S_k$
 - Select the best k of these to be the new current states. That is, select best k from $S_1 \cup S_2 \cup \dots \cup S_k$
- Is this the same as running hill-climbing search k times with different random initial states?
- No. Notice that in local beam search we select from among ALL successors. Thus, there is some **sharing** of information across the k searches.
- However, this can cause the k states to become more narrowly focused on certain areas of the search space and ignore exploring other areas. i.e lack of *diversity* in the k states.
- **Stochastic beam search**: choose successors with prob. proportional to the successor's value. This helps increase diversity.

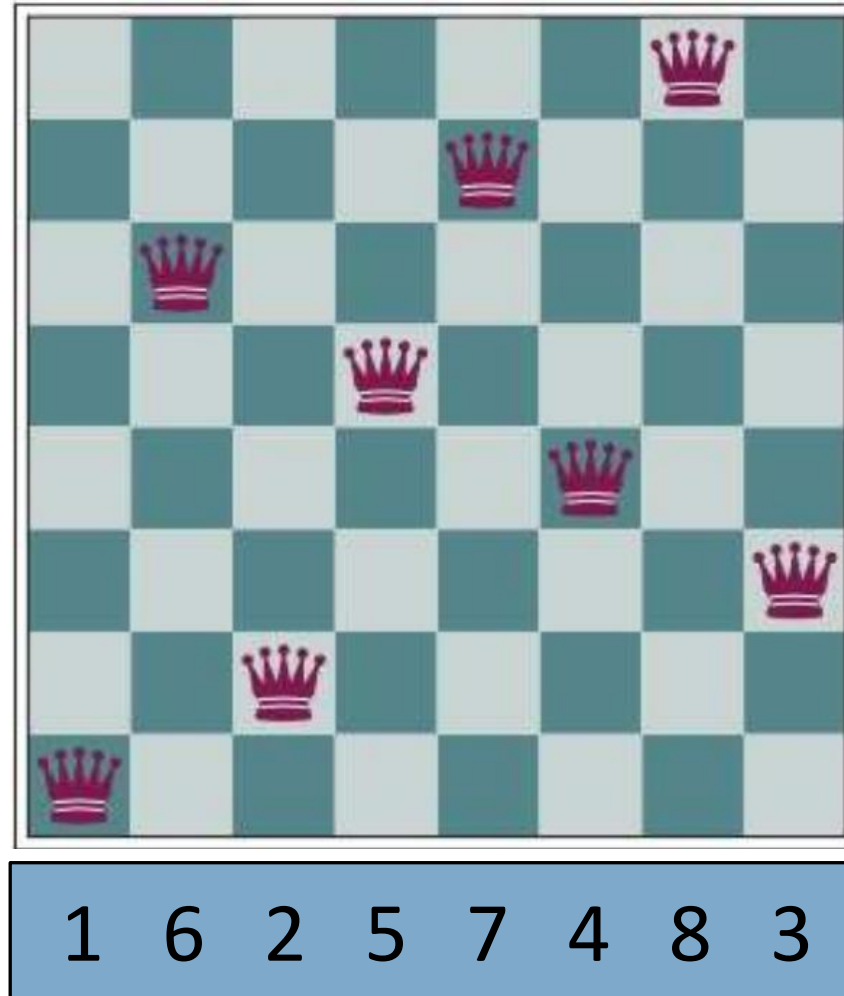
Evolutionary Algorithms

Evolutionary Algorithms [1]

- Stochastic beam search: choose successors with prob. proportional to the successor's value. This helps increase diversity.
- Evolutionary algorithms are variants of stochastic beam search.
- Basic idea:
 - Maintain **current population of states** in memory. Initialized at random.
 - For N iterations do:
 - Select the fittest (best) individuals from the current population to produce the next generation (successor states). This next generation will become the current population.
 - Return the best individual (state) from the final population.

Genetic Algorithm on 8-queens [1]

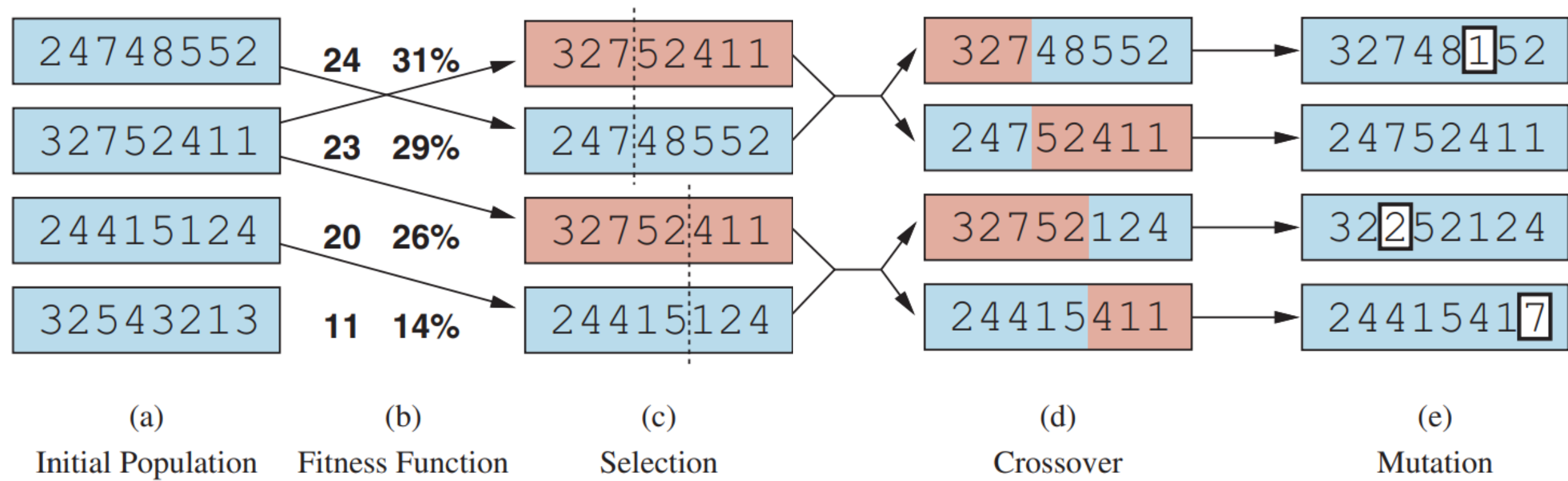
- Consider the 8-queens problem again where we represent states by an 8-digit sequence of numbers denoting the position of the 8-queens.



Genetic Algorithm on 8-queens [2]

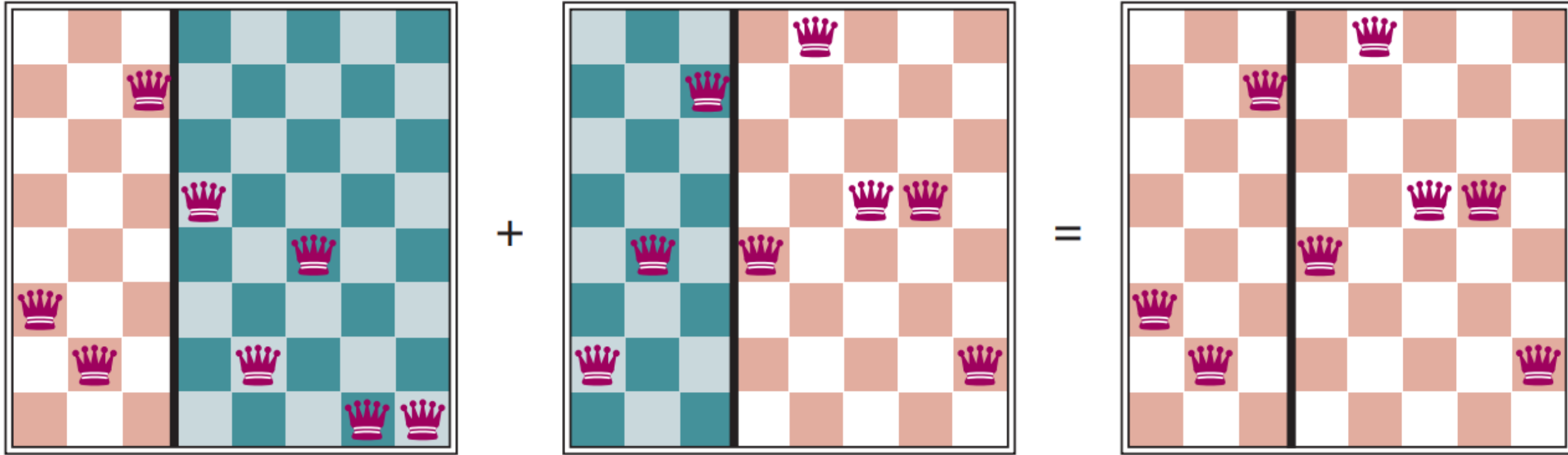
- Consider the 8-queens problem again where we represent states by an 8-digit sequence of numbers denoting the position of the 8-queens.
- We want to run genetic algorithm search with:
 - Population size of 4.
 - Fitness/value function = number of nonattacking pairs of queens. Optimal value is $8*7/2 = 28$ (handshake formula)
 - Use 2 parents to produce 2 successors using random crossovers. Thus, we need to select 4 parents from current population to produce 4 successors.
 - Select the 4 parents with prob. proportional to fitness.
 - Apply random mutation to a single position with probability p .

Genetic Algorithm on 8-queens [3]

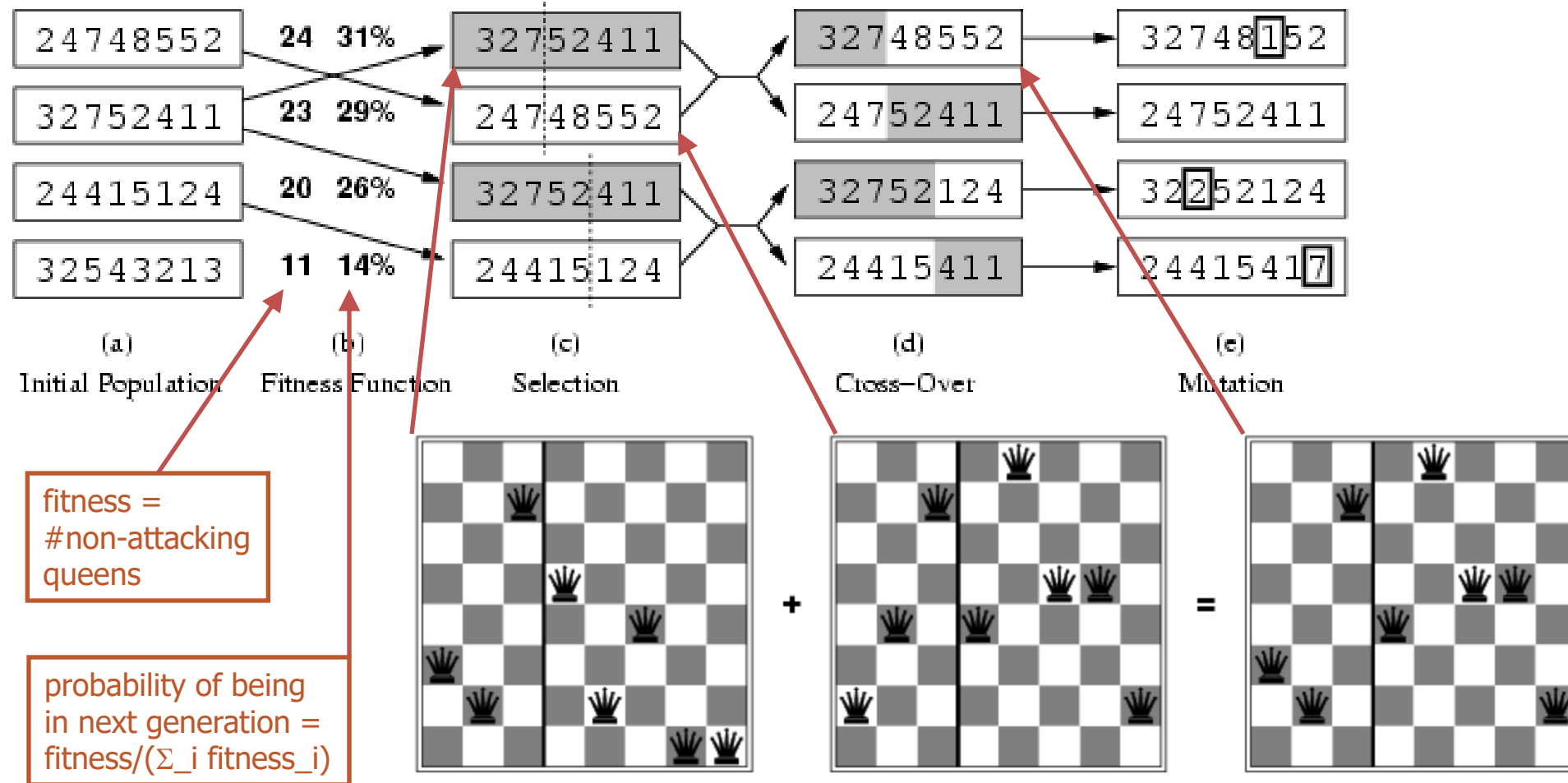


- Fitness function (value): number of non-attacking pairs of queens (min = 0, max = $8 \times 7/2 = 28$)
 - $24/(24+23+20+11) = 31\%$
 - $23/(24+23+20+11) = 29\%$; etc.
- These steps are performed in each iteration.
- Or until some condition is met; e.g. an individual with a high fitness is produced.

Genetic Algorithm on 8-queens [4]



- The crossover step corresponds to combining two halves of the board.
- In general, how much crossover helps depends on the problem and state representation
- Not clear if genetic algorithms are popular because they are actually superior on specific tasks, or if they are popularized due to popularity of evolution theory.
- Used in circuit layout and job scheduling.



Fitness function: #non-attacking queen pairs. min = 0, max = $8 \times 7/2 = 28$

$\sum_i \text{fitness}_i = 24 + 23 + 20 + 11 = 78$

$P(\text{pick child}_1 \text{ for next gen.}) = \text{fitness}_1 / (\sum_i \text{fitness}_i) = 24/78 = 31\%$

$P(\text{pick child}_2 \text{ for next gen.}) = \text{fitness}_2 / (\sum_i \text{fitness}_i) = 23/78 = 29\%$; etc

How to convert a fitness value into a probability of being in the next generation.

```

function GENETIC-ALGORITHM(population, fitness) returns an individual
  repeat
    weights  $\leftarrow$  WEIGHTED-BY(population, fitness)
    population2  $\leftarrow$  empty list
    for i = 1 to SIZE(population) do
      parent1, parent2  $\leftarrow$  WEIGHTED-RANDOM-CHOICES(population, weights, 2)
      child  $\leftarrow$  REPRODUCE(parent1, parent2)
      if (small random probability) then child  $\leftarrow$  MUTATE(child)
      add child to population2
    population  $\leftarrow$  population2
  until some individual is fit enough, or enough time has elapsed
  return the best individual in population, according to fitness

```

```

function REPRODUCE(parent1, parent2) returns an individual
  n  $\leftarrow$  LENGTH(parent1)
  c  $\leftarrow$  random number from 1 to n
  return APPEND(SUBSTRING(parent1, 1, c), SUBSTRING(parent2, c + 1, n))

```

- This pseudocode from figure 4.8 in textbook implements a particular genetic algorithm setting. So we can modify it to another setting; e.g. to use 3 parents instead of 2 with a different reproduce definition.

Evolutionary Algorithms Design Choices

- There are many forms of evolutionary algorithms. They vary by tweaking the following:
 - Size of population to maintain in memory. A population consists of individuals (states).
 - How to represent individuals (states). That is, how to formulate/represent a state for the problem.
 - How many parents (states) to combine to produce a successor. Common is 2 parents to produce a successor.
 - How to select the parents from the current population. Randomness is injected in the selection process to encourage diversity.
 - Select individuals with prob. proportional to fitness (value).
 - Select n individuals uniform-randomly, then select the most fit from this random set.
 - How to combine the parents to produce a successor.
 - Common approach with 2 parents, is to randomly select a **crossover** point to split the 2 parents and combine the two splits of the parents.
 - Likelihood of a produced successor to form a random mutation. E.g. a bit/ is randomly flipped, or a value is altered.
 - After generating successors, what should be the next generation of the population.
 - Can include some of the high-score individuals from the current population in the next generation
 - Can remove low-score individuals from the produced successors
 - Other modifications possible. It is more of a design/artistic approach.

Summary

Summary

- Many configuration and optimization problems can be formulated as local search
- Advantage of local search algorithms:
 - Low memory upkeep.
 - Can reach reasonable solutions fast.
- General families of algorithms:
 - Random-walk
 - Hill-climbing (discrete); gradient-descent (continuous)
 - Simulated annealing
 - Local beam search: multiple interaction searches
 - Genetic algorithms: break and recombine states
- May get stuck in local optima, ridges, shoulders, flat areas, etc. Vulnerable to *bad* initial random state.